PIATTAFORMA DI SVILUPPO DI IA UNIFICATA LOCAL-FIRST CON MEMORIA VETTORIALE INTEGRATA, ORCHESTRAZIONE MULTI-MODELLO E ARCHITETTURA CLIENT-SERVER DISTRIBUITA

Domanda di Brevetto

Richiedente / Cessionario: ODAXAI S.R.L.
Sede legale: Via Calle Pace 40, 35031 Abano Terme (PD), Italia
Partita IVA: IT05755990289 **Codice fiscale:** 05755990289 **N. Registro Imprese:** PD – 490222
PEC (posta elettronica certificata): odaxai@pec.it
Inventore/i: Nicolò Savioli
Data di priorità: 29 gennaio 2026
Riferimento interno (docket): ODX–ODAXSTUDIO–PPA–001

Nota: La presente specifica omette intenzionalmente il codice sorgente. La divulgazione è espressa in termini di sistemi e metodi, con molteplici realizzazioni non limitative.

RIASSUNTO

[0000001] Viene divulgata una piattaforma local-first per lo sviluppo di intelligenza artificiale. In esempi di realizzazione, un'applicazione host nativa multi-piattaforma ospita un'interfaccia unificata e orchestra molteplici servizi locali e di rete comprendenti: (i) un server di inferenza LLM, (ii) un server di editor di codice integrato e (iii) un server frontend web. La piattaforma comprende inoltre un *Global Memory Layer* persistente implementato utilizzando un database vettoriale locale con deduplicazione basata su hash e recupero ibrido semantico/parole chiave, un gestore multi-modello categorizzato che supporta la selezione del modello a runtime e l'hot-swapping, chat in streaming ed esecuzione di agenti tramite Server-Sent Events, e un parser semantico del codice sensibile al linguaggio per il chunking di documenti allineato ai confini. Tutti i dati sensibili dell'utente (ad es. codice sorgente, documenti, prompt, embedding e cronologia chat) sono elaborati interamente sul dispositivo dell'utente, abilitando il funzionamento offline e la preservazione della riservatezza. In alcune realizzazioni, la piattaforma fornisce inoltre un livello opzionale di gamification e condivisione che calcola statistiche d'uso aggregate (ad es. token, throughput, tempo di risposta) e, a seguito di opt-in dell'utente, pubblica statistiche selezionate su un feed/profilo di attività condivisibile senza esporre i prompt sottostanti o gli artefatti locali.

CAMPO TECNICO

[0000002] La presente divulgazione riguarda ambienti di sviluppo per l'intelligenza artificiale (IA) e, più in particolare, piattaforme local-first, atte a preservare la riservatezza, che integrano modelli linguistici di grandi dimensioni (LLM), memoria vettoriale distribuita su NAS, editor di codice e processamento multimodale all'interno di un'architettura client-server unificata.

DESCRIZIONE DELLO STATO DELLA TECNICA

Contesto tecnico

[0000003] I flussi di lavoro di sviluppo assistiti da IA si basano sempre più su assistenti conversazionali, completamento del codice, analisi di documenti e generazione aumentata dal recupero (RAG). Molti sistemi esistenti sono forniti come servizi cloud, applicazioni web o plug-in per editor che delegano l'inferenza e la memoria a server esterni.

Limitazioni della tecnica nota

[0000004] La dipendenza dal cloud introduce rischi per la riservatezza e la sicurezza, poiché i prompt dell'utente, il codice sorgente, i documenti e il contesto di progetto possono essere trasmessi a infrastrutture di terze parti, e la disponibilità può essere limitata dalla connettività di rete, da interruzioni del servizio, da limiti di frequenza e da costi di abbonamento.

[0000005] I flussi di lavoro esistenti sono inoltre frammentati. Gli sviluppatori passano regolarmente tra applicazioni separate (interfacce chat, editor di codice, motori di ricerca, database vettoriali e dashboard API). Tale frammentazione aumenta il carico cognitivo e richiede ripetuti copia-incolla manuali, ripetuta fornitura di contesto e stato incoerente tra strumenti.

[0000006] Molti assistenti non dispongono di memoria locale persistente. Essi possono non mantenere una rappresentazione accurata e duratura del codebase dell'utente, dei documenti, delle conversazioni storiche, della terminologia di dominio o delle preferenze tra sessioni. Di conseguenza, gli utenti ripetono la progettazione dei prompt e ricaricano il contesto.

[0000007] Molti strumenti basati su cloud offrono flessibilità limitata nella scelta e nel deployment del modello. Gli utenti possono non essere in grado di operare offline, di selezionare modelli diversi per classi di compiti differenti (ad es. coding vs. ragionamento), di utilizzare modelli quantizzati ottimizzati per l'hardware disponibile o di adottare modelli fine-tuned e proprietari per compiti specifici di dominio.

[0000008] Infine, gli strumenti basati sul web spesso non dispongono di un'integrazione profonda con il sistema operativo, inclusa una solida orchestrazione di processi locali, gestione del ciclo di vita, monitoraggio dello stato di salute dei servizi, networking sicuro solo su loopback e funzionalità UI native quali scorciatoie a livello di sistema e menu nativi.

SOMMARIO DELL'INVENZIONE

[0000009] Il sistema divulgato, qui denominato *OdaxStudio*, fornisce una piattaforma unificata local-first per lo sviluppo di IA che affronta le limitazioni sopra descritte mediante orchestrazione coordinata dei servizi, memoria vettoriale persistente distribuita su NAS, gestione multi-modello categorizzata tramite server di inferenza dedicato (*OdaxBox*), parsing semantico del codice, interazione in streaming in tempo reale ed esecuzione on-device a tutela della riservatezza.

[00010] In esempi di realizzazione, OdaxStudio comprende: (i) un'applicazione client multi-

piattaforma che incorpora un'interfaccia utente web locale; (ii) un server di inferenza dedicato (*OdaxBox*) che gestisce modelli LLM e orchestrazione; (iii) un ProcessManager che avvia, monitora e arresta molteplici processi comprendenti server di editor di codice integrato e server frontend web; (iii) un Global Memory Layer che utilizza un database vettoriale locale con deduplicazione basata su hash; e (iv) recupero ibrido che combina ricerca di similarità semantica con filtro basato su parole chiave, abilitando la generazione aumentata dal recupero preservando la riservatezza.

[00011] Ulteriori realizzazioni comprendono (i) un catalogo di modelli multi-categoria (ad es. `coding`, `chat`, `thinking` e una categoria proprietaria riservata), (ii) un gestore locale di download dei modelli per modelli quantizzati in formato GGUF, (iii) caricamento dei modelli a runtime e hot-swapping senza riavviare l'applicazione host, (iv) un parser semantico del codice sensibile al linguaggio che suddivide il codice ai confini di funzioni/classi quando disponibili e (v) un flusso di lavoro agente in stile ReAct i cui passaggi intermedi sono trasmessi alla UI tramite Server-Sent Events (SSE) per visualizzazione in tempo reale.

[00012] In ulteriori realizzazioni, OdaxStudio comprende un *livello di gamification e condivisione* che strumenta le operazioni locali di inferenza e recupero per calcolare telemetria d'uso aggregata (ad es. token generati, conteggio di inferenze, throughput e tempo di risposta) e presenta tale telemetria in un pannello profilo utente. A seguito di un'azione esplicita di opt-in, il sistema può creare un "post di attività" sanitizzato contenente solo statistiche aggregate selezionate (e facoltativamente classe hardware o identificatore del modello) e pubblicare il post su una pagina utente e/o feed (ad es. una timeline in stile Twitter) senza trasmettere codice sorgente, contenuti di documenti, prompt o embedding.

- Predefinito privato: la telemetria è memorizzata localmente e la condivisione è disabilitata salvo abilitazione da parte dell'utente.
- Minimizzazione: vengono pubblicate solo metriche aggregate; il testo e i file sottostanti rimangono sul dispositivo.
- Controlli di condivisione: la configurazione per singolo post può includere o escludere campi (token, token/sec, nome del modello, classe hardware).

[00013] Le tecniche divulgate forniscono una piattaforma pratica e sicura per lo sviluppo locale di IA mantenendo gli artefatti sensibili sul dispositivo, abilitando il funzionamento offline, riducendo la latenza e consolidando strumenti di sviluppo altrimenti frammentati in un ambiente coerente.

BREVE DESCRIZIONE DEI DISEGNI

[00014] FIG. 1 illustra un esempio di architettura di sistema in cui un'applicazione client OdaxStudio comunica con un server di inferenza OdaxBox, un NAS per lo storage distribuito del Global Memory Layer, un server IDE integrato e servizi di rete locali.

[00015] FIG. 2 illustra un esempio di pipeline locale di ingestione di documenti dalla scansione delle directory fino al parsing specifico per tipo, chunking, deduplicazione, generazione di embedding e memorizzazione vettoriale.

[00016] FIG. 3 illustra un esempio di flusso di lavoro di parsing semantico del codice comprendente rilevamento del linguaggio, applicazione di pattern di confine, chunking sensibile ai confini e comportamento di fallback.

[00017] FIG. 4 illustra un esempio di procedura di recupero ibrido che combina ricerca di similarità

semantica vettoriale con filtro per parole chiave, seguita da unione e ri-ordinamento per produrre contesto RAG.

[00018] FIG. 5 illustra un esempio di sistema di gestione multi-modello con metadati dei modelli categorizzati, tracciamento del download locale, verifica, caricamento a runtime e hot-swapping.

[00019] FIG. 6 illustra un esempio di flusso di esecuzione di un agente ReAct con classificazione dei passaggi e streaming SSE verso una UI del pannello di ragionamento.

[00020] FIG. 7 illustra un esempio di architettura di chat in streaming in cui uno stream di token SSE aggiorna incrementalmente la UI mentre le conversazioni persistono localmente.

[00021] FIG. 8 illustra un esempio di confine di riservatezza in cui i dati sensibili dell'utente rimangono sul dispositivo, con connessioni esterne opzionali limitate al download dei modelli e/o a query di ricerca web pubbliche.

[00022] FIG. 9 illustra un esempio di interfaccia chat di OdaxStudio che fornisce selezione del modello e risposte in streaming.

[00023] FIG. 10 illustra un esempio di UI del pannello di ragionamento per visualizzare in tempo reale i passaggi di esecuzione dell'agente.

[00024] FIG. 11 illustra un esempio di visualizzatore di documenti integrato nella piattaforma.

[00025] FIG. 12 illustra un esempio di pannello profilo utente e statistiche che consente all'utente di visualizzare l'uso dei token in tempo reale e di attivare, tramite opt-in, la condivisione dell'attività.

[00026] FIG. 13 illustra un esempio di feed di attività in stile Twitter e pagina utente che visualizza post di statistiche di utilizzo condivise.

[00027] FIG. 14 illustra l'architettura distribuita OdaxBox-NAS-OdaxStudio che mostra la comunicazione tra il server di inferenza OdaxBox, il NAS per lo storage del vector database, e i client OdaxStudio sulla rete LAN locale.

DESCRIZIONE DETTAGLIATA DELL'INVENZIONE

Definizioni e panoramica

[00028] Salvo indicazione contraria, i termini "comprendente," "incluso," e "avente" sono intesi come inclusivi. Le realizzazioni qui descritte sono illustrative e non limitative. Una piattaforma "local-first" si riferisce a un sistema in cui il calcolo e lo storage primari avvengono su un dispositivo controllato dall'utente, con eventuali interazioni di rete opzionali limitate a operazioni non sensibili.

[00029] OdaxStudio può essere implementato come un'applicazione client multi-piattaforma che incorpora un'interfaccia web servita localmente. L'interfaccia web può fornire chat, esplorazione di documenti, controlli di indicizzazione, selezione del modello e visualizzazione degli agenti. I servizi possono comunicare tramite rete locale protetta (LAN) per garantire sicurezza e prevenire l'accesso esterno non autorizzato.

Architettura del sistema

[00030] FIG. 1 mostra un esempio di architettura. Un'applicazione client OdaxStudio (101) esegue su un dispositivo utente e include un'interfaccia web (102) e un ProcessManager (103). Il ProcessManager coordina la comunicazione con: un server di inferenza dedicato OdaxBox (105) che gestisce modelli LLM, un NAS di rete (108) che ospita il Global Memory Layer con database vettoriale distribuito, un frontend web (104) e un server IDE integrato (106). L'architettura supporta deployment sia in configurazione single-machine che in configurazione distribuita su LAN. Servizi esterni opzionali (109) possono

includere hosting di modelli per download e/o ricerca web solo snippet.

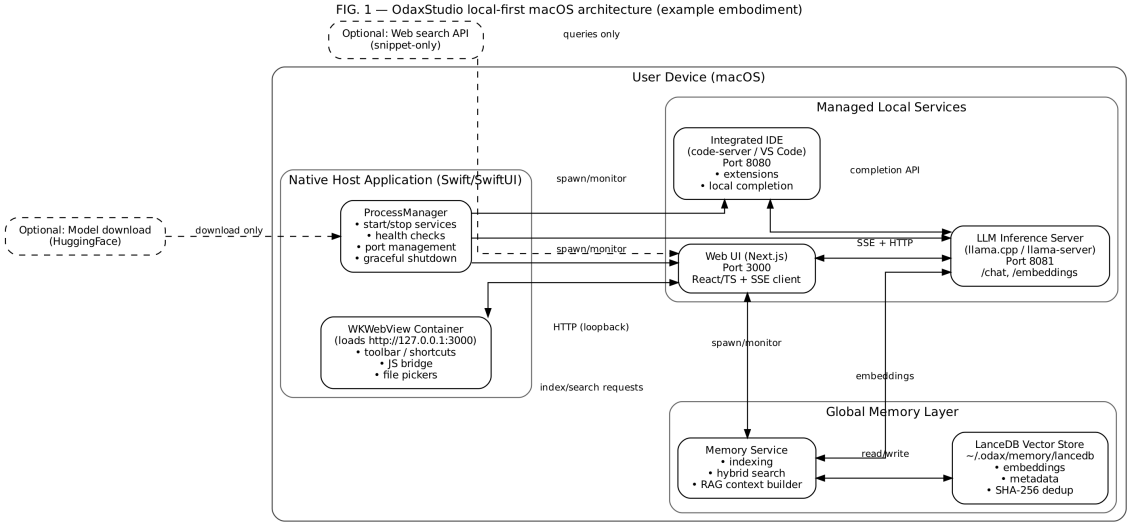

Figura 1. Architettura OdaxStudio local-first distribuita con OdaxBox e NAS (esempio di realizzazione).

Orchestrazione di processi nativi e gestione dei servizi

[00031] In esempi di realizzazione, il ProcessManager mantiene una sequenza di avvio deterministica. Il ProcessManager può creare e verificare directory richieste (ad es. `~/.odax/models`, `~/.odax/memory`, `~/.odax/downloads`); selezionare un modello disponibile; avviare il server di inferenza; verificare la prontezza del servizio tramite probe delle porte o endpoint di health; avviare il server IDE; e infine avviare il frontend web.

[00032] Il ProcessManager può implementare uno shutdown graceful utilizzando segnali del sistema operativo. Ad esempio, all'uscita dell'applicazione può inviare un segnale di terminazione a ciascun processo figlio, attendere un timeout configurabile e opzionalmente escalare a interruzione o terminazione forzata se i processi rimangono attivi. Il monitoraggio dello stato di salute può includere controlli periodici per la disponibilità delle porte, la vitalità dei processi e la responsività alle richieste.

[00033] In alcune realizzazioni, il ProcessManager inoltre esegue allocazione delle porte per evitare collisioni, mantiene un modello di stato per il rendering della UI (ad es. "Avvio del motore IA...") e registra diagnostica e log in una directory locale per il troubleshooting.

Global Memory Layer

[00034] Il Global Memory Layer fornisce memoria locale persistente per la piattaforma. Può indicizzare directory selezionate dall'utente e memorizzare rappresentazioni a livello di chunk in un database vettoriale locale, insieme a metadati quali percorso del file, indice del chunk, linguaggio, tipo di file e timestamp. Ciò abilita il recupero di contesto rilevante per assistenza al codice, Q&A su documenti e flussi di lavoro degli agenti senza caricare dati su un servizio remoto.

[00035] FIG. 2 mostra un esempio di pipeline di ingestione. Uno scanner di file enumera file sotto una o più directory applicando esclusioni (ad es. `.git`, `node_modules`), soglie di dimensione e filtri per estensione. Un rilevatore di tipo classifica ciascun file come codice, testo, PDF o immagine, e instrada il file a un parser specifico per tipo. Il contenuto parsato è suddiviso in chunk e quindi deduplicato utilizzando un hash del file, dopodiché gli embedding sono generati tramite un endpoint di embedding locale e memorizzati in un database vettoriale.

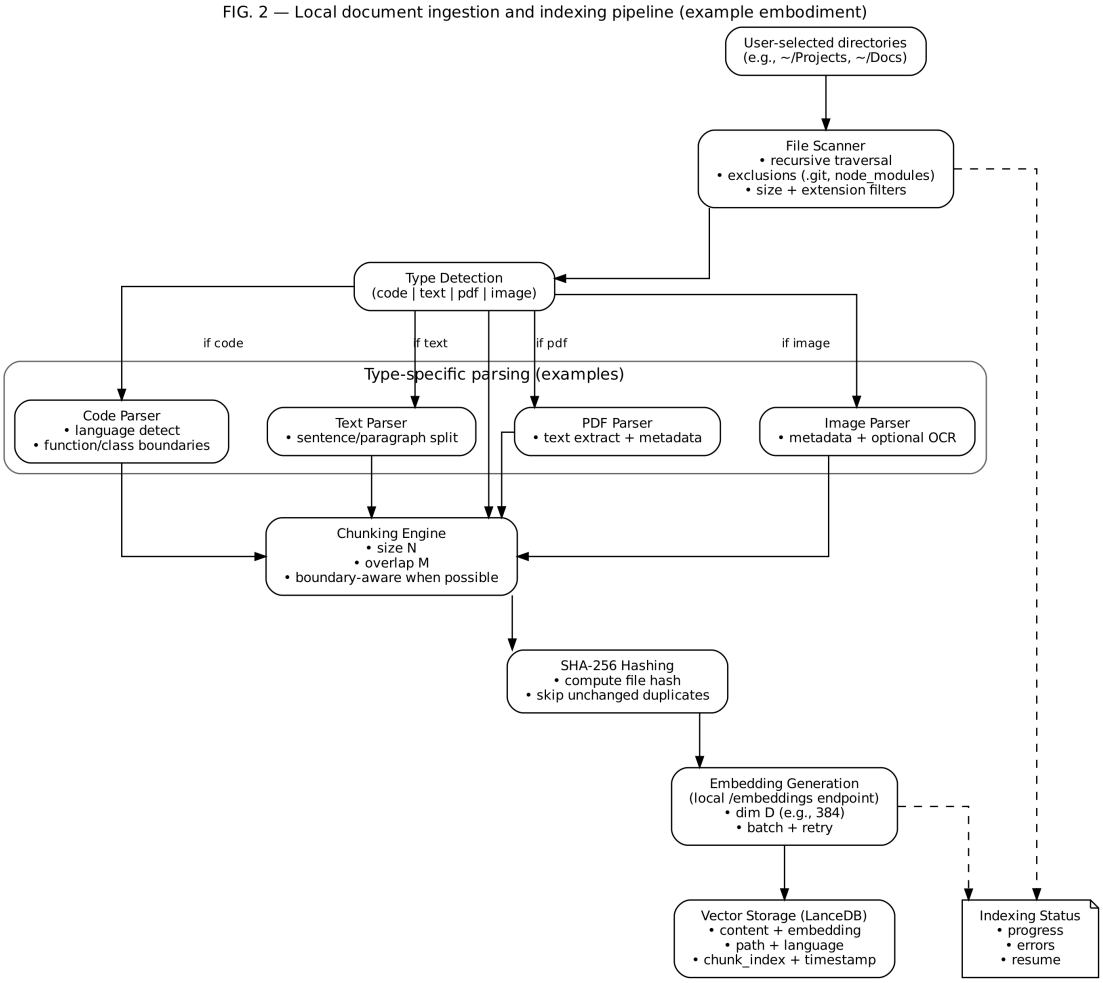

Figura 2. Pipeline locale di ingestione e indicizzazione di documenti (esempio di realizzazione).

Deduplicazione basata su hash e aggiornamenti incrementali

[00036] Per ridurre l'indicizzazione ridondante e conservare il calcolo, il sistema può calcolare un hash crittografico (ad es. SHA-256) per ciascun file. Se un hash di file corrisponde a un record già indicizzato, il sistema può saltare la re-indicizzazione del file. In alcune realizzazioni, il sistema mantiene una mappatura dal percorso del file all'hash più recente e supporta aggiornamenti incrementali quando i file cambiano.

[00037] La deduplicazione può essere eseguita a granularità di file e/o a granularità di chunk. In una realizzazione, il sistema memorizza l'hash del file in ciascun record di chunk e lo utilizza per eliminare e sostituire tutti i chunk appartenenti a un file modificato durante la re-indicizzazione.

Parsing semantico del codice e chunking

[00038] FIG. 3 mostra un esempio di flusso di lavoro di parsing semantico del codice. Il sistema rileva il linguaggio di programmazione utilizzando mappatura delle estensioni dei file ed euristiche opzionali. Pattern di confine specifici per linguaggio sono applicati per identificare confini di chunk corrispondenti a unità semantiche quali funzioni, classi, moduli e/o definizioni di metodi. I blocchi identificati sono impacchettati in chunk secondo una dimensione massima e overlap, preservando i confini quando possibile.

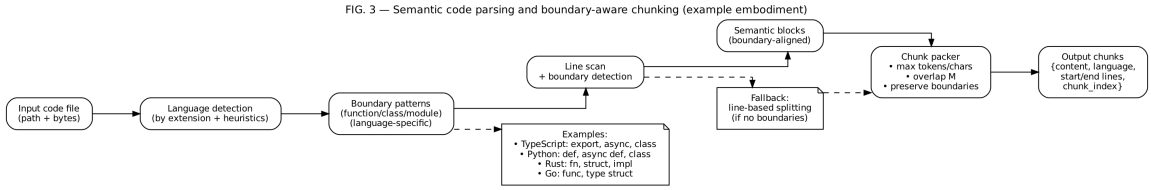

Figura 3. Parsing semantico del codice e chunking sensibile ai confini (esempio di realizzazione).

[00039] Il chunking semantico migliora il recupero e il completamento del codice preservando unità coerenti di codice. Ad esempio, quando si indicizzano file TypeScript, i confini possono includere funzioni esportate, dichiarazioni di classe e assegnazioni di arrow-function. Per Python, i confini possono includere dichiarazioni `def`, `async def` e `class`. Per Rust, i confini possono includere blocchi `fn`, `struct` e `impl`. Pattern simili possono essere definiti per altri linguaggi.
[00040] Se non vengono rilevati confini, il sistema può ricadere su chunking basato su righe. In alcune realizzazioni, l'overlap dei chunk è applicato in modo che una finestra di contesto appaia in chunk adiacenti, migliorando le prestazioni di recupero downstream.

Recupero ibrido e costruzione del contesto RAG

[00041] FIG. 4 illustra un esempio di algoritmo di recupero ibrido. Un percorso semantico genera un embedding per la query ed esegue ricerca di similarità vettoriale contro il database vettoriale per ottenere candidati top-K. In parallelo, un percorso per parole chiave tokenizza la query ed esegue filtro per parole chiave (ad es. corrispondenza SQL `LIKE`, BM25 o altro scoring lessicale) per ottenere candidati aggiuntivi. I risultati sono uniti, deduplicati, normalizzati a range di score comparabili e ri-ordinati con vincoli di diversità.

FIG. 4 — Hybrid semantic + keyword search with merging and reranking (example embodiment)

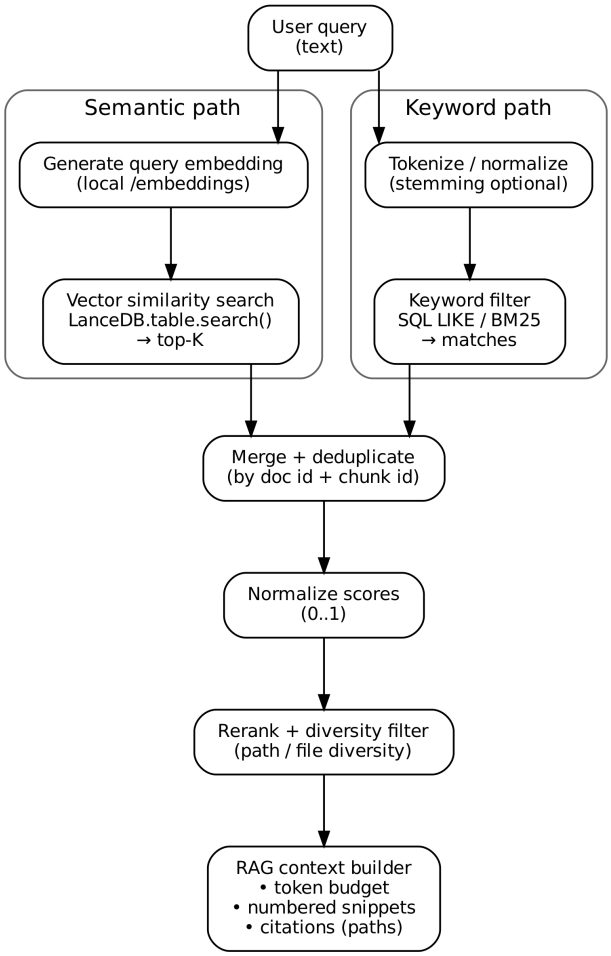

Figura 4. Ricerca ibrida semantica + parole chiave con unione e ri-ordinamento (esempio di realizzazione).

[00042] I risultati ri-ordinati possono essere formattati come contesto aumentato dal recupero per una chiamata LLM downstream. In un esempio di realizzazione, il costruttore di contesto seleziona snippet sotto un budget di token, numera ciascuno snippet e include metadati sorgente quali percorso del file e range di righe per supportare la tracciabilità.

Gestione multi-categoria dei modelli e hot-swapping

[00043] OdaxStudio può mantenere un catalogo di modelli comprendente multiple categorie, ad esempio: coding, chat, thinking (ragionamento profondo) e una categoria proprietaria riservata. Ciascuna voce del catalogo può includere metadati quali un identificatore del repository, nome del file, formato di quantizzazione, dimensione attesa e descrizione.

[00044] FIG. 5 illustra un esempio di flusso di gestione dei modelli. A seguito di una richiesta di download, il sistema scarica un file di modello quantizzato (ad es. GGUF) in un archivio di modelli locale. Il progresso del download può essere tracciato monitorando la crescita della dimensione del file e persistendo metadati di progresso (ad es. in un file JSON). Al completamento, il sistema verifica e registra il modello. Il caricatore runtime quindi carica il modello nel server di inferenza. Un'API di hot-swap può scaricare un modello corrente e caricare un modello diverso senza riavviare l'applicazione host.

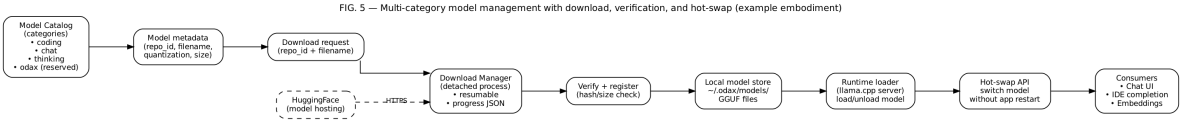

Figura 5. Gestione multi-categoria dei modelli con download, verifica e hot-swap (esempio di realizzazione).

[00045] I formati di quantizzazione possono includere, a titolo di esempio, formati a 4 bit (ad es. varianti Q4) e formati a bit inferiori (ad es. varianti a 2 bit), abilitando modelli più grandi a funzionare su hardware consumer. Il sistema può esporre parametri di configurazione quali lunghezza del contesto, numero di layer GPU e dimensioni del batch a seconda dell'hardware.

Esecuzione agente ReAct con visualizzazione in tempo reale

[00046] OdaxStudio può implementare un flusso di lavoro agente che intercala ragionamento e azione. In una realizzazione, un agente è modellato come una sequenza di passaggi classificati in tipi quali *thinking*, *search*, *visit* e *answer*. Ciascun passaggio può produrre eventi strutturati trasmessi al client tramite SSE.

[00047] FIG. 6 illustra un esempio di flusso ReAct con visualizzazione dei passaggi. La UI può visualizzare un pannello di ragionamento che si aggiorna in tempo reale, mostrando tipi di passaggio con icone, colori e timestamp distinti. Ciò fornisce trasparenza nell'esecuzione dell'agente e può migliorare il debugging e la fiducia dell'utente.

FIG. 6 — ReAct agent with step classification and SSE visualization (example embodiment)

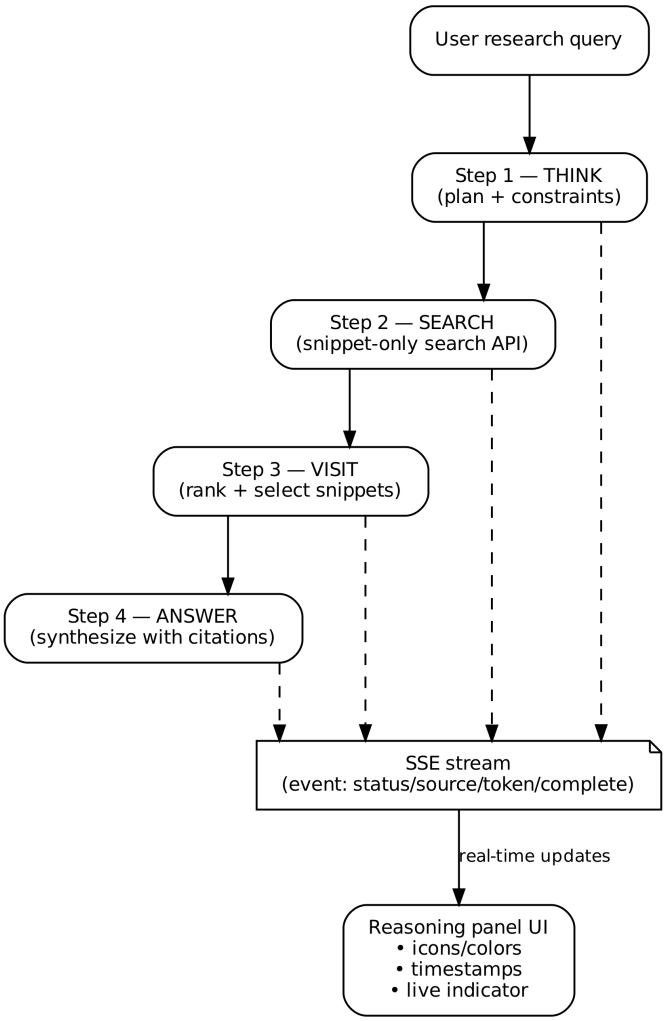

Figura 6. Agente ReAct con classificazione dei passaggi e visualizzazione SSE (esempio di realizzazione).

[00048] In alcune realizzazioni, un passaggio di ricerca interroga un'API di ricerca web in modalità solo snippet (cioè senza scraping di pagine complete). L'agente può quindi selezionare e citare snippet quando produce una risposta. In altre realizzazioni, l'agente opera rigorosamente offline utilizzando solo il Global Memory Layer.

Chat in streaming con persistenza locale

[00049] La piattaforma può fornire un'interfaccia di chat in streaming. In una realizzazione, i messaggi dell'utente sono inviati tramite HTTP POST a una route API locale che fa da proxy alle richieste al server di inferenza. Il server di inferenza restituisce token utilizzando SSE, e la UI aggiorna incrementalmente il messaggio dell'assistente visualizzato man mano che i token arrivano.

[00050] FIG. 7 illustra un esempio di flusso di chat in streaming. Lo stato della conversazione può essere mantenuto in un store client (ad es. un contenitore di stato) e persistito localmente (ad es. utilizzando storage del browser). Il sistema può generare automaticamente titoli delle conversazioni, tracciare la selezione del modello per conversazione e mantenere lo stato di streaming separatamente dallo stato persistente.

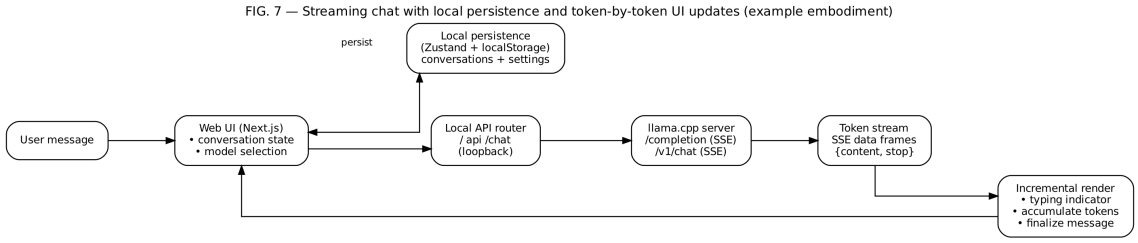

Figura 7. Chat in streaming con persistenza locale e aggiornamenti UI token-per-token (esempio di realizzazione).

IDE integrato con completamento IA locale

[00051] In esempi di realizzazione, la piattaforma incorpora un server IDE completo (ad es. code-server) per fornire un editor di codice integrato. L'IDE può caricare un'estensione locale che richiede completamenti dal server di inferenza locale. Ciò abilita il completamento inline del codice senza trasmettere codice a un servizio remoto.

[00052] In una realizzazione, il server IDE ascolta solo su interfacce loopback. L'autenticazione può essere disabilitata per una configurazione puramente locale, o alternativamente abilitata utilizzando token e/o integrazione con keychain del sistema operativo.

Riservatezza e funzionamento offline

[00053] FIG. 8 illustra un esempio di confine di riservatezza. In realizzazioni preferite, artefatti sensibili (codice sorgente, documenti, prompt, embedding e cronologia chat) rimangono entro il confine di esecuzione locale del dispositivo dell'utente. Interazioni di rete opzionali possono essere limitate al download di artefatti di modelli e/o all'emissione di query di ricerca web pubbliche, e possono essere disabilitate completamente per ottenere un funzionamento completamente offline.

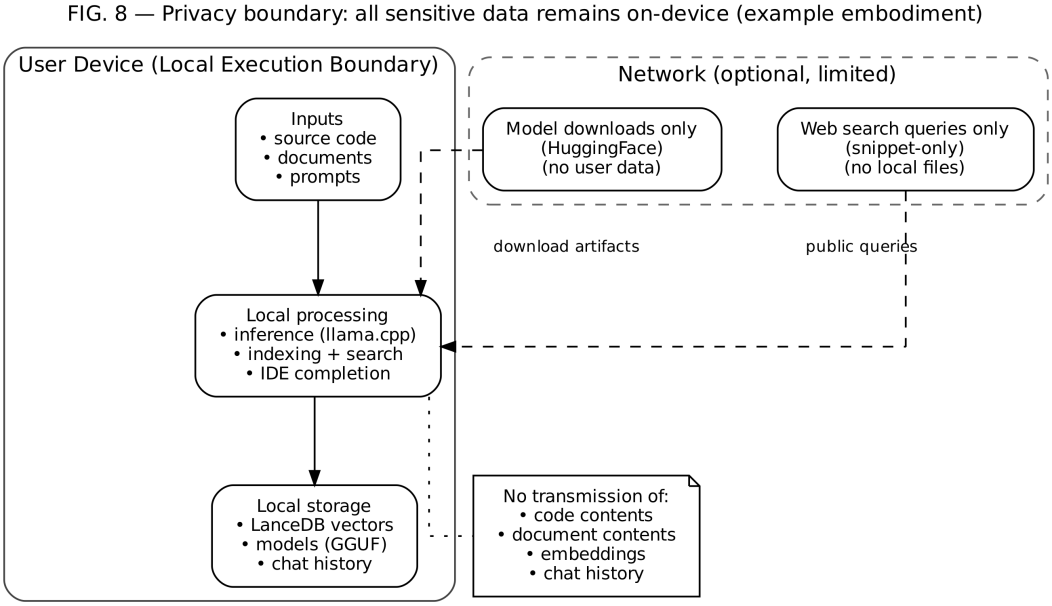

Figura 8. Confine di riservatezza: tutti i dati sensibili rimangono sul dispositivo (esempio di realizzazione).

Architettura distribuita OdaxBox-NAS per deployment LAN

[00054] In realizzazioni avanzate, il sistema può essere deployato in una configurazione distribuita client-server su rete locale (LAN). In questa configurazione, un server dedicato denominato *OdaxBox* ospita il motore di inferenza LLM e l'orchestrazione dei modelli, mentre un dispositivo di storage di rete (NAS) ospita il Global Memory Layer con il database vettoriale distribuito. Molteplici client OdaxStudio possono connettersi simultaneamente al server OdaxBox tramite la rete locale.

[00055] FIG. 14 illustra questa architettura distribuita. Il server OdaxBox (componente centrale) gestisce: (i) inferenza LLM tramite modelli quantizzati locali, (ii) orchestrazione e hot-swapping dei modelli, (iii) generazione di embedding, e (iv) esposizione di API per i client. Il NAS ospita: (i) il database vettoriale distribuito, (ii) documenti e codice sorgente indicizzati, (iii) embedding persistenti, e (iv) indici per il recupero ibrido.

[00056] Il server OdaxBox comunica bidirezionalmente con il NAS: scrive embedding e metadati durante l'indicizzazione e legge chunk rilevanti durante il recupero RAG. I client OdaxStudio inviano query e prompt al server OdaxBox e ricevono risposte in streaming tramite SSE. Opzionalmente, i client possono accedere direttamente al NAS per browsing e gestione dei documenti.

[00057] Questa architettura fornisce: (i) scalabilità orizzontale (molteplici client possono condividere lo stesso server di inferenza), (ii) centralizzazione della memoria (un singolo database vettoriale condiviso), (iii) isolamento della rete (tutto il traffico rimane sulla LAN locale senza connessioni Internet), (iv) ottimizzazione delle risorse (hardware dedicato per inferenza può essere più potente di singoli dispositivi client), e (v) gestione semplificata (modelli e indici sono mantenuti centralmente).

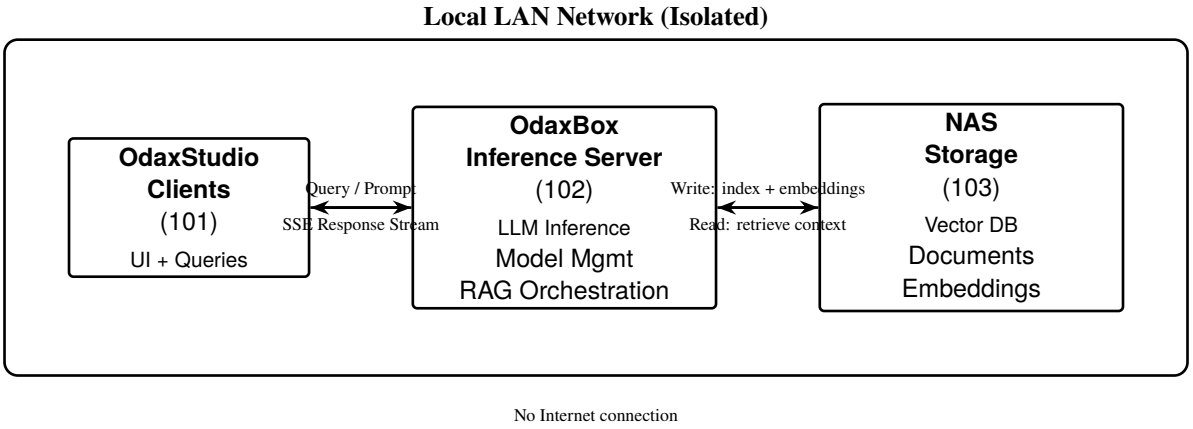

Figura 9. FIG. 14 – Distributed system architecture. OdaxStudio clients (101) send queries to OdaxBox inference server (102). OdaxBox writes indexes/embeddings to NAS storage (103) and reads back retrieved context for RAG. All traffic stays inside an isolated local LAN.

Gamification, telemetria d'uso e profili condivisibili

[00058] In alcune realizzazioni, la piattaforma fornisce un meccanismo di feedback *gamificato* inteso ad aumentare l'engagement e fornire visibilità nelle prestazioni local-first. Durante l'inferenza on-device, il proxy di inferenza e/o il server LLM emette contatori runtime inclusi conteggio token, token-per-secondo, latenza di risposta e conteggio di inferenze. Un aggregatore di telemetria locale memorizza questi contatori in un datastore locale (ad es. all'interno della sandbox dell'applicazione o directory utente) e li espone alla UI per la visualizzazione in un pannello profilo.

[00059] A seguito di un'azione esplicita dell'utente (ad es. abilitare un toggle "Condividi Attività" e selezionare un template di post), il sistema genera un payload di attività sanitizzato contenente solo statistiche aggregate, opzionalmente un identificatore del modello e opzionalmente un descrittore hardware grossolano (ad es. classe CPU). Il payload esclude esplicitamente prompt, testo di documenti recuperati, codice sorgente, percorsi di file e embedding. In una realizzazione, il payload di attività è pubblicato su una pagina utente online e feed renderizzato in un formato timeline sociale, consentendo così agli utenti di condividere risultati di prestazioni (ad es. token generati a un dato throughput) senza divulgare informazioni sensibili.

[00060] In ulteriori realizzazioni, la pubblicazione supporta controlli di riservatezza inclusi opt-in per post, redazione a livello di campo e sincronizzazione ritardata. Il sistema può inoltre calcolare metriche derivate (ad es. streak giornaliere, distribuzione d'uso del modello e categorie di compiti) localmente e condividere solo riepiloghi aggregati.

Esempi di schermate dell'interfaccia utente

[00061] FIG. 9 illustra un esempio di interfaccia chat, inclusa selezione del modello e risposte in streaming. FIG. 10 illustra un esempio di UI del pannello di ragionamento per visualizzare passaggi di esecuzione dell'agente. FIG. 11 illustra un esempio di visualizzatore di documenti integrato. FIG. 12 illustra un esempio di pannello statistiche profilo utente con controlli di condivisione opt-in. FIG. 13 illustra un esempio di pagina feed pubblica o di team che renderizza post di attività condivisi (ad es. card di utilizzo token).

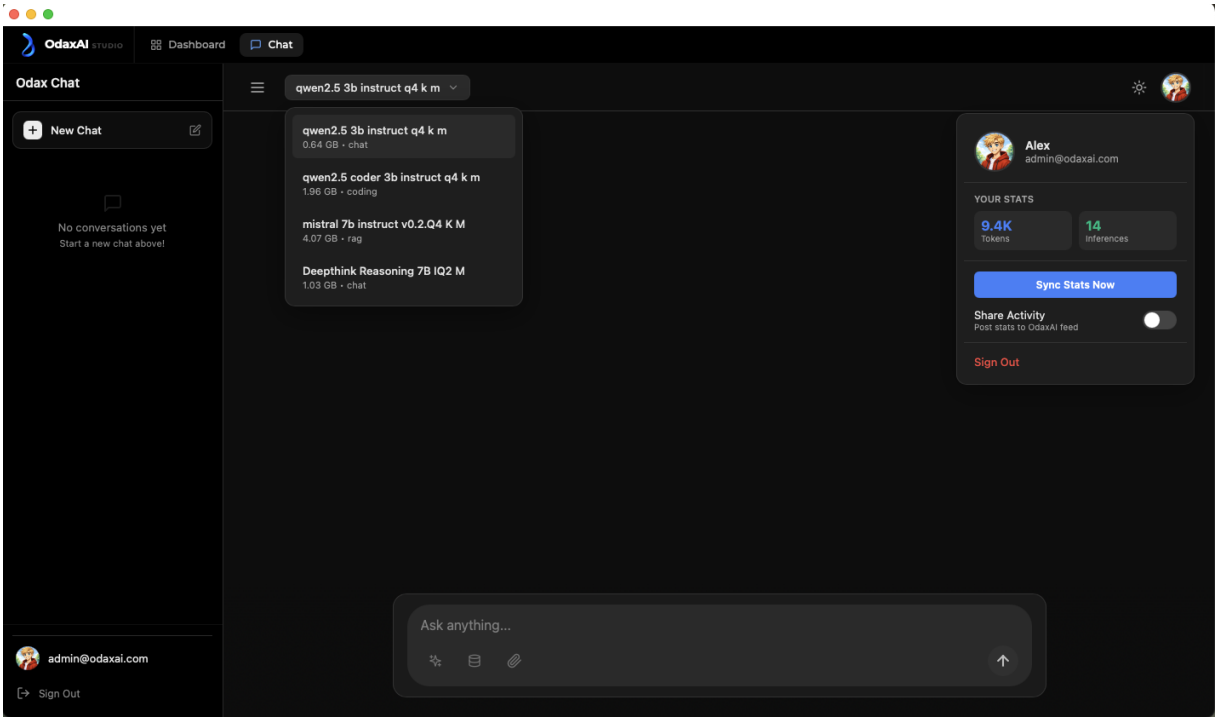

Figura 10. Esempio di interfaccia chat OdaxStudio (screenshot illustrativo).

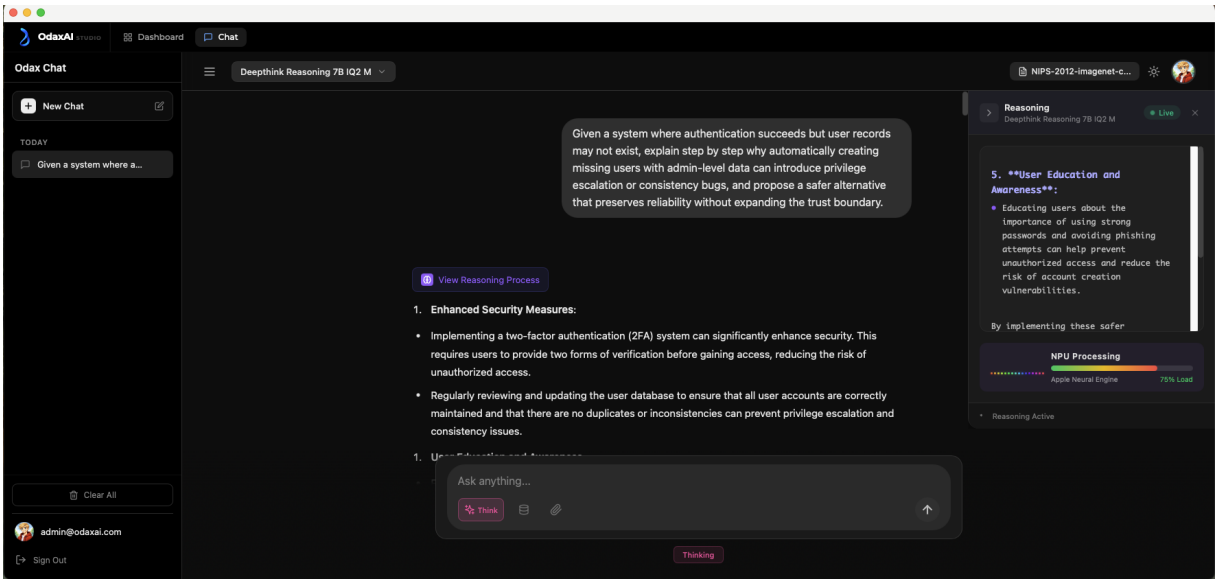

Figura 11. Esempio di pannello di ragionamento OdaxStudio con visualizzazione dei passaggi dell'agente (screenshot illustrativo).

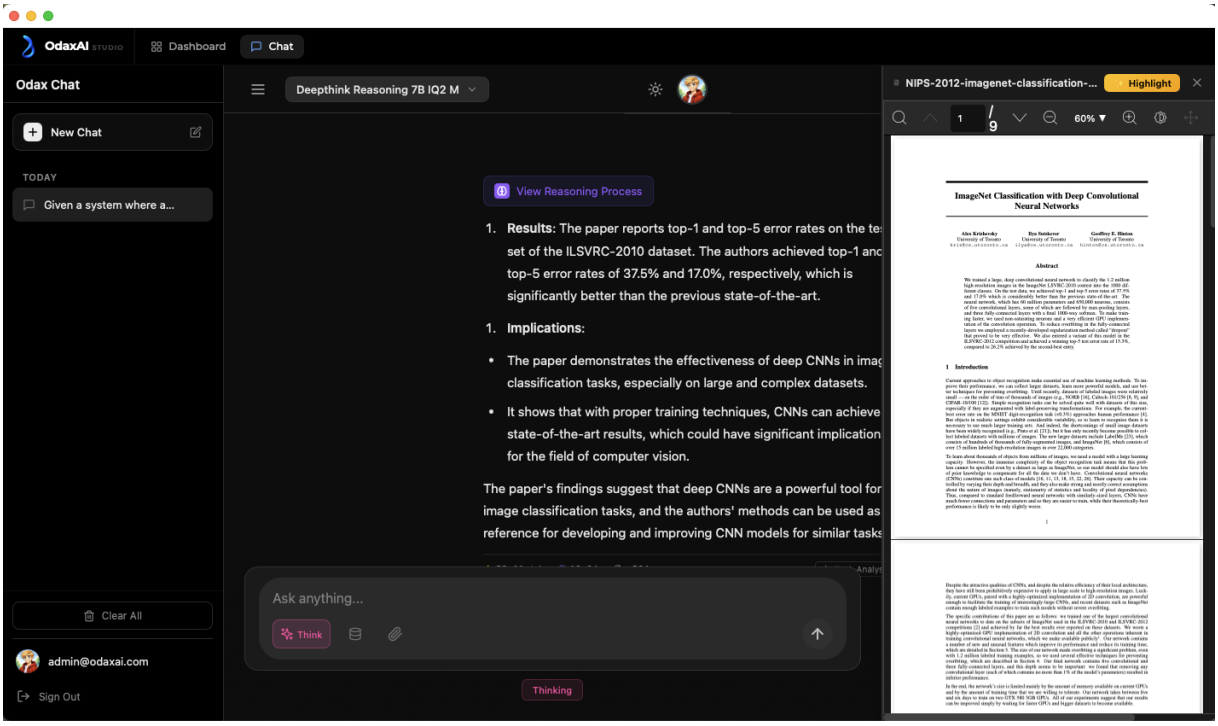

Figura 12. Esempio di visualizzatore di documenti integrato OdaxStudio (screenshot illustrativo).

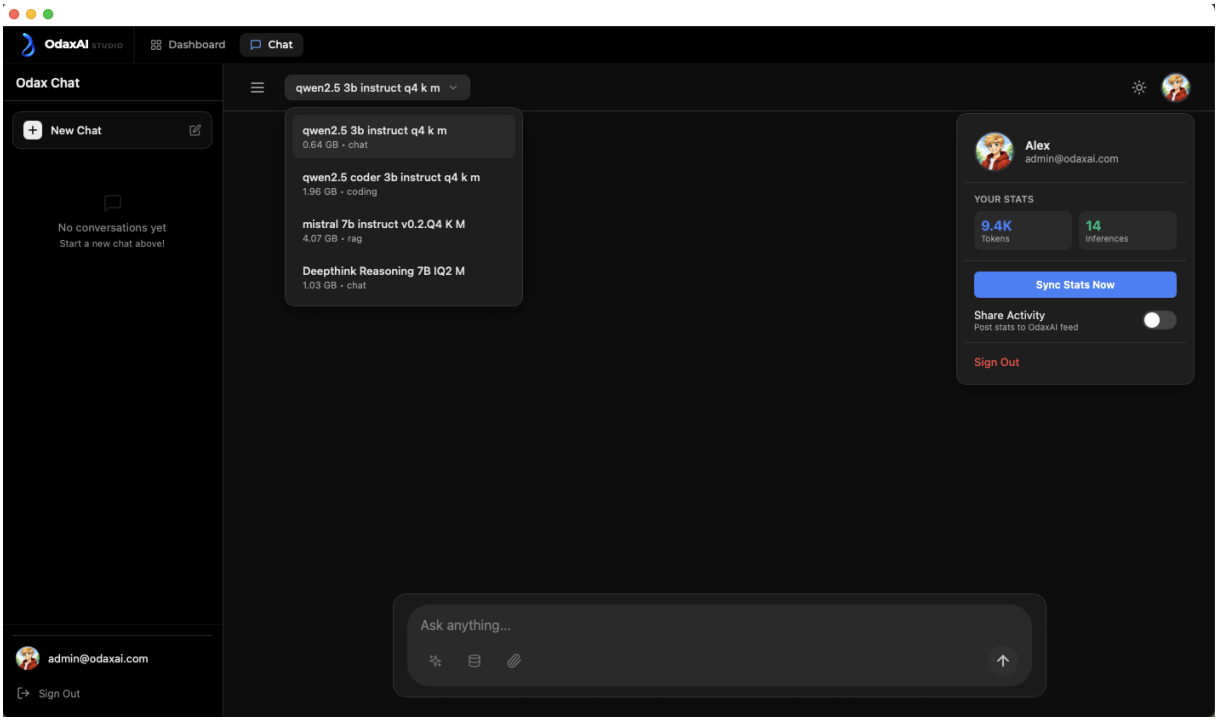

Figura 13. Esempio di pannello statistiche utente OdaxStudio che mostra utilizzo token e controlli di condivisione opt-in (screenshot illustrativo).

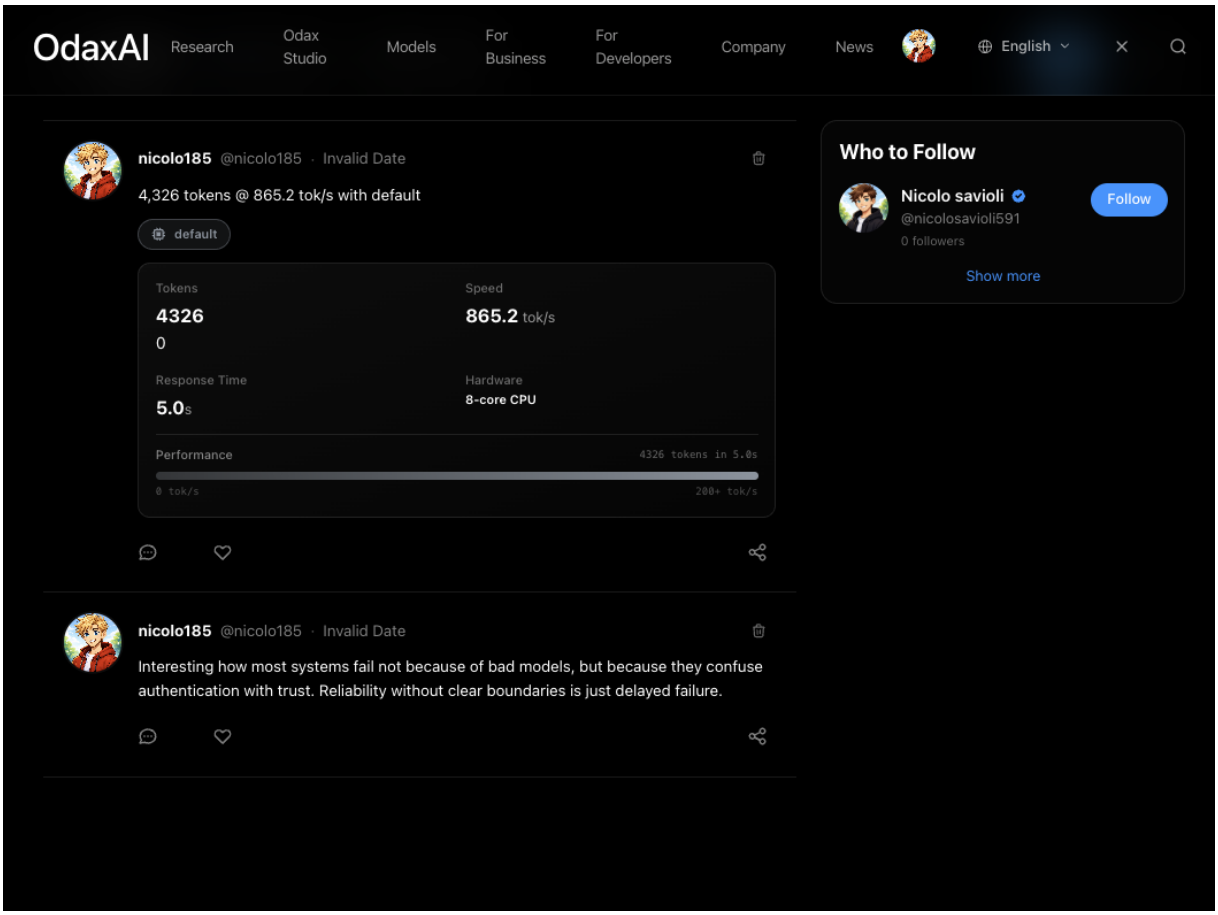

Figura 14. Esempio di feed di attività online/pagina utente OdaxStudio che visualizza post di statistiche di utilizzo condivise (screenshot illustrativo).

Realizzazioni alternative

[00062] La piattaforma è progettata per essere multi-piattaforma e può essere implementata su sistemi operativi diversi (Windows, macOS, Linux) utilizzando contenitori host nativi e gestori di servizi appropriati. Il database vettoriale può essere sostituito (ad es. store vettoriali basati su SQLite, PostgreSQL con pgvector, o altri motori embedded/distribuiti). Il backend di inferenza può essere sostituito (ad es. runtime di inferenza locale o cloud alternativi) e il formato del modello può variare. L'architettura supporta deployment sia centralizzato (tutto su un dispositivo) che distribuito (client-server con NAS).

[00063] Il Global Memory Layer può indicizzare modalità aggiuntive (ad es. immagini, audio e database strutturati). Il modulo di recupero ibrido può incorporare segnali di ranking aggiuntivi quali recentezza, ponderazione del percorso del file, tag specifici del progetto, feedback dell'utente o ri-ordinatori appresi.

[00064] L'architettura OdaxBox-NAS può essere estesa con: (i) clustering multi-server per alta disponibilità, (ii) replicazione del database vettoriale su molteplici NAS, (iii) bilanciamento del carico tra molteplici istanze OdaxBox, (iv) sincronizzazione cloud opzionale per backup off-site (con crittografia end-to-end), e (v) supporto per deployment ibrido (parte on-premise, parte cloud).

RIVENDICAZIONI (NON LIMITATIVE)

[00065] Le seguenti rivendicazioni sono fornite come esempi non limitativi per chiarire aspetti della divulgazione. In una successiva domanda non provvisoria, le rivendicazioni possono essere modificate, aggiunte o rimosse.

[00066] 1. Metodo attuato da elaboratore per fornire una piattaforma di sviluppo di intelligenza artificiale local-first, comprendente: (a) ricevere, da uno o più client OdaxStudio, una richiesta che include almeno una query testuale e, opzionalmente, riferimenti a codice sorgente e/o documenti locali; (b) selezionare, mediante un gestore multi-modello, un modello tra una pluralità di modelli disponibili, in funzione di una categoria di compito, preferibilmente includente almeno `coding`, `chat` e `thinking`; (c) indicizzare e mantenere una memoria persistente comprendente almeno un database vettoriale, memorizzato localmente o su un dispositivo di storage di rete (NAS), in cui sono archiviati embeddings e metadati relativi a file e chunk; (d) eseguire recupero ibrido per costruire contesto per generazione aumentata dal recupero (RAG), comprendendo (i) ricerca di similarità vettoriale su embeddings e (ii) filtro lessicale basato su parole chiave, e unire e ri-ordinare risultati ottenuti; (e) inviare al server di inferenza OdaxBox la query e il contesto RAG, ed eseguire inferenza LLM per generare una risposta; (f) trasmettere la risposta ai client OdaxStudio in modalità streaming, preferibilmente tramite Server-Sent Events (SSE); in cui (i) le operazioni di indicizzazione, recupero e inferenza avvengono entro un confine di esecuzione locale e/o di rete locale isolata e (ii) dati sensibili non sono trasmessi a server esterni.

[00067] 2. Il metodo secondo la rivendicazione 1, in cui l'indicizzazione comprende: (a) scansionare una o più directory applicando esclusioni configurabili e filtri per estensione e dimensione; (b) rilevare tipi di file includenti almeno codice, testo, PDF e immagini; e (c) instradare ciascun file a un parser specifico per tipo prima della generazione degli embeddings.

[00068] 3. Il metodo secondo la rivendicazione 2, in cui il chunking di file di codice comprende applicare rilevamento di confini semantici speci-

fici per linguaggio per produrre chunk allineati a unità selezionate tra funzioni, metodi, classi e moduli, e, in assenza di confini rilevati, applicare un fallback a chunking basato su righe con overlap configurabile.

[00069] 4. Il metodo secondo la rivendicazione 2, in cui la deduplicazione comprende calcolare un hash crittografico per ciascun file e saltare la re-indicizzazione quando l'hash corrisponde a un record già indicizzato, e, quando l'hash differisce, sostituire i chunk associati al file modificato.

[00070] 5. Il metodo secondo la rivendicazione 1, in cui il gestore multi-modello comprende: (a) scaricare modelli quantizzati in un archivio locale o di rete, tracciando lo stato di download e verificando l'integrità del file scaricato; e (b) effettuare hot-swapping tra modelli caricati, comprendendo scaricare un modello corrente e caricare un modello differente senza interrompere l'interfaccia utente dei client.

[00071] 6. Il metodo secondo la rivendicazione 1, in cui il server OdaxBox coordina una configurazione distribuita su LAN comprendente un NAS che ospita il database vettoriale, e in cui il server OdaxBox esegue scrittura di embeddings e metadati sul NAS durante l'indicizzazione e lettura di chunk rilevanti dal NAS durante il recupero RAG.

[00072] 7. Il metodo secondo la rivendicazione 1, in cui la piattaforma comprende un flusso di lavoro agente che intercala ragionamento e azione, e in cui passaggi intermedi dell'agente sono serializzati come eventi e trasmessi in streaming ai client per visualizzazione in tempo reale.

[00073] 8. Il metodo secondo la rivendicazione 1, comprendente raccogliere telemetria d'uso aggregata relativa ad almeno uno tra numero di token generati, throughput e latenza, memorizzare la telemetria localmente e pubblicare, a seguito di opt-in esplicito dell'utente, un payload sanitizzato che esclude contenuto dei prompt e contenuto dei file locali.

[00074] 9. Sistema per fornire una piattaforma di sviluppo di intelligenza artificiale local-first, comprendente uno o più processori e una memoria contenente istruzioni che, quando eseguite, configurano il sistema ad eseguire il metodo secondo una qualsiasi delle rivendicazioni 1–8, includendo un server di inferenza OdaxBox, uno o più client OdaxStudio e, opzionalmente, un dispositivo NAS per memoria vettoriale persistente.

[00075] 10. Supporto non transitorio leggibile da elaboratore che memorizza istruzioni che, quando eseguite da uno o più processori, fanno eseguire il metodo secondo una qualsiasi delle rivendicazioni 1–8.

GLOSSARIO

[00076] GGUF: un formato di file per modelli transformer quantizzati per inferenza locale efficiente. **Database vettoriale:** un archivio embedded che supporta ricerca di similarità su vettori di embedding. **RAG:** generazione aumentata dal recupero utilizzando snippet recuperati come contesto per un modello. **SSE:** Server-Sent Events, un meccanismo di streaming HTTP per aggiornamenti server-to-client. **ReAct:** un pattern agente che intercala passaggi di ragionamento e passaggi di azione/strumento. **Token:** un'unità di generazione di testo utilizzata per misurare la lunghezza input/output del modello. **Token/sec:** una metrica di throughput che indica un tasso di generazione di token. **Telemetria d'uso:** statistiche aggregate calcolate localmente che descrivono il comportamento runtime (ad es. token generati, latenza, throughput) senza esporre il contenuto sottostante. **Feed di attività:** una UI in stile timeline che ren-

derizza post di utilizzo selezionati dall'utente e sanitizzati.

NICOLÒ
SAVIOLI
29.01.2026
10:00:52
GMT+01:00